

Guía de Oportunidades para Toda la Vida 89 — Desarrollador de Software

Versión: 2.0 maestra de trabajo controlada

Idioma: Español neutro de América Latina (es-419)

Referencia principal de EE. UU.: O*NET-SOC 15-1252.00 — Software Developers

Comparación de Canadá: NOC 21232 — Software developers and programmers

Comparación de Colombia: CUOC 25120 — Desarrolladores de software

Fecha de revisión: 2026-08-22

Fuente inglesa congelada: blob
ef6a140a6dae98e48560a2de40365053bd914755

Qué es esta carrera

Un desarrollador de software analiza necesidades, diseña y construye software, prueba y mejora soluciones, documenta decisiones y apoya el producto durante cambios y operación. Puede trabajar en aplicaciones de escritorio o móviles, servicios cloud, componentes embebidos, sistemas internos, APIs, procesamiento de datos, plataformas o utilidades especializadas.

El título es amplio. Algunas personas desarrollan funciones de aplicaciones; otras trabajan en sistemas distribuidos, cloud, herramientas DevOps, software embebido, sistemas empresariales, servicios de datos o infraestructura. A largo plazo importa menos memorizar muchos lenguajes que comprender requisitos, escribir soluciones mantenibles, probar supuestos, depurar fallas, proteger datos, colaborar y aprender tecnologías nuevas.

Estados Unidos tiene una referencia directa en **O*NET-SOC 15-1252.00 — Software Developers**, Bright Outlook y actualizada en 2026. Canadá corresponde a **NOC 21232 — Software developers and programmers**. Colombia tiene una correspondencia directa en **CUOC 25120 — Desarrolladores de software**.

Por qué sigue siendo una carrera fuerte

El software sostiene finanzas, logística, salud, gobierno, comunicaciones, transporte, manufactura, ciberseguridad, medios y educación.

La proyección nacional de EE. UU. muestra **16% de crecimiento entre 2024 y 2034** y alrededor de **115,200 vacantes anuales**, incluyendo crecimiento y reemplazo. Son cifras ocupacionales nacionales, no una garantía de empleo individual.

Un desarrollador sólido combina:

- definición de problemas;
- fundamentos de programación;
- estructuras de datos y algoritmos;
- diseño de software;
- APIs y persistencia;
- pruebas;
- debugging y observabilidad;
- seguridad y privacidad;
- control de versiones;
- documentación;
- disciplina de liberación;
- uso responsable de automatización e IA.

Familias de funciones

Desarrollador de aplicaciones

Construye aplicaciones para usuarios o procesos de negocio y puede trabajar en interfaz, servicios y datos.

Desarrollador back-end o de servicios

Se enfoca en APIs, lógica de negocio, persistencia, mensajería, caché, integración y confiabilidad.

Desarrollador de plataforma/infraestructura

Construye tooling, plataformas internas, automatización y software cercano a infraestructura.

Desarrollador móvil

Trabaja con aplicaciones móviles nativas o multiplataforma, permisos, almacenamiento, red y ciclo de vida del dispositivo.

Desarrollador de sistemas/embebido

Trabaja más cerca de sistemas operativos, dispositivos, hardware, redes o código sensible al rendimiento.

Empiece por los requisitos

Antes del código, aclare:

1. ¿Qué problema resolvemos?
2. ¿Qué comportamiento se requiere?
3. ¿Qué queda fuera de alcance?

4. ¿Qué inputs/outputs existen?
5. ¿Qué requisitos de rendimiento, disponibilidad, privacidad o seguridad aplican?
6. ¿Qué sistemas e interfaces se afectan?
7. ¿Qué evidencia demostrará que funciona?
8. ¿Quién aprueba requisito y liberación?

Si los requisitos son contradictorios, documente y escale la ambigüedad. No invente silenciosamente comportamiento después de implementar.

Diseño antes de implementación

Según el riesgo, piense en:

- componente responsable;
- datos a almacenar;
- contrato de API/interfaz;
- fallas de dependencias;
- compatibilidad;
- migración/rollback;
- controles de acceso;
- observabilidad;
- estrategia de pruebas.

Las decisiones de arquitectura pueden pertenecer a ingenieros senior, arquitectos o equipos de plataforma/seguridad. Contribuya evidencia sin asumir autoridad no asignada.

Fundamentos de programación

Aprenda conceptos transferibles:

- variables y tipos;
- control de flujo;
- funciones/métodos;
- módulos/paquetes;
- colecciones;
- errores/excepciones;
- interfaces/abstracciones;
- conceptos orientados a objetos y funcionales cuando correspondan;
- comportamiento asíncrono/concurrente;
- pruebas y debugging.

No intente dominar todos los lenguajes al mismo tiempo. Aprenda un stack con suficiente profundidad para comprender ejecución real.

Estructuras de datos y algoritmos

Comprenda listas/arrays, mapas/diccionarios, sets, stacks/queues, árboles/grafos a nivel apropiado, búsqueda/ordenamiento, iteración y complejidad de tiempo/espacio.

No todos los puestos requieren algoritmos avanzados, pero comprender complejidad ayuda a evitar diseños ineficientes.

Tecnologías actuales

O*NET muestra para 2025:

- Python **29%**;
- AWS **26%**;
- Java **25%**;
- SQL **24%**;
- JavaScript **20%**;
- Azure **19%**;
- Kubernetes y Git **14%**;
- RESTful API, React y Docker **13%**;
- C# **12%**;
- C++ y Angular **10%**;
- CSS y Linux **9%**;
- Jenkins CI, HTML y TypeScript **8%**;
- Node.js, JIRA, GitHub y NoSQL **7%**;
- PostgreSQL, Terraform, Kafka y C **6%**;
- Spring Boot, Go y Spring Framework **5%**.

Son señales de ofertas, no requisitos universales.

APIs y contratos

Las interfaces deben definir inputs, outputs, errores, compatibilidad y versionado. Pueden ser REST/HTTP, RPC, mensajería/eventos, SDKs, esquemas de bases de datos o formatos de archivos.

No cambie silenciosamente un contrato consumido por otros sistemas.

Bases de datos y persistencia

Conceptos útiles:

- tablas/documentos;
- claves/relaciones;
- índices;
- transacciones;

- consistencia;
- migraciones;
- conexiones;
- caché;
- retención;
- responsabilidades de backup/recovery.

Use consultas parametrizadas o mecanismos ORM aprobados. No concatene input no confiable en SQL.

Autenticación y autorización

Autenticación establece identidad. Autorización determina permisos. Siga el modelo aprobado, mínimo privilegio y autorización del lado del servidor. Ocultar controles de UI no protege un recurso.

No debilita autorización solo para hacer que una función pase una prueba.

Git y code review

Use commits claros, branches según política, pull/merge requests, revisión entre pares y resolución de conflictos.

Nunca confirme credenciales, API keys, tokens, certificados o secretos. Si un secreto entra al historial, siga rotación e incidente; borrar una línea visible no necesariamente lo elimina.

Pruebas y QA

Pueden incluir:

- unitarias;
- integración;
- contrato/API;
- componentes;
- end-to-end;
- regresión;
- rendimiento;
- accesibilidad;
- seguridad dentro de autorización.

Que una suite pase es evidencia, no prueba de software sin defectos o seguro.

Debugging y observabilidad

Flujo útil:

1. reproducir;
2. identificar versión/ambiente;
3. reducir el problema;
4. revisar logs, métricas, traces, inputs y estado;
5. formular hipótesis;
6. probarla;
7. aplicar una corrección segura;
8. agregar/ajustar pruebas;
9. retestar y monitorear.

Use logs estructurados, métricas, tracing donde aplique, IDs de correlación, health checks y alertas. No registre passwords, tokens, datos completos de pago o información personal innecesaria.

Errores y resiliencia

Planifique input inválido, timeouts, fallas parciales, servicios no disponibles, duplicados, retries y degradación. Use retries limitados/backoff e idempotencia cuando corresponda. Reintentar todo ciegamente puede amplificar una falla.

Concurrencia y asincronía

Según el rol, comprenda race conditions, sincronización, estado compartido, colas, orden, eventual consistency, cancelación/timeouts y procesamiento duplicado.

Código asíncrono no es automáticamente correcto solo porque compila.

Dependencias y supply chain

Use dependencias necesarias y confiables, revise advisories, versiones y licencias cuando corresponda, pruebe upgrades y evite código desconocido o paquetes abandonados.

Configuración y secretos

Proteja passwords de bases de datos, API keys, credenciales cloud, OAuth secrets, signing/encryption keys y certificados. Use sistemas aprobados de secrets/configuration.

CI/CD

Un pipeline confiable identifica commit/versión, dependencias, resultados de pruebas, artifacts, ambiente y approvals/gates. No evite gates obligatorios solo porque una liberación manual sea más rápida.

Deploy, cambio y rollback

Antes de producción, comprenda versión, configuración, migraciones, dependencias, monitoreo, rollback/forward-fix, compatibilidad de datos y quién tiene autoridad de release.

Tener acceso técnico no equivale a tener autoridad de producción.

Cloud y responsabilidad compartida

AWS, Azure, Docker y Kubernetes son frecuentes. Los servicios cloud no asumen automáticamente todas las responsabilidades de identidad, aplicación, configuración, secretos, datos y código. Siga el modelo del servicio y la arquitectura del empleador.

Rendimiento

Mida antes de optimizar. Revise CPU/memoria, I/O, red, queries, caché, concurrencia, pools de conexiones, payloads y dependencias. Las pruebas de carga deben realizarse dentro de límites autorizados.

Desarrollo seguro

Prácticas relevantes incluyen validación, manejo seguro de datos/output, autenticación/autorización, queries parametrizadas, defaults seguros, secrets, dependencias, logging, code review y pruebas de seguridad autorizadas.

NIST SSDF y OWASP son referencias útiles. No otorgan permiso para atacar o hacer pentest de sistemas.

Privacidad y accesibilidad

Use solo datos necesarios para fines aprobados y siga reglas de acceso, retención, borrado, datos de prueba y logging. No invente requisitos legales; escale a la función responsable.

Según el producto, considere teclado, estructuras semánticas, labels, foco, contraste, errores comprensibles y compatibilidad con tecnologías de asistencia. Un scanner automático no demuestra cumplimiento legal.

IA responsable

La IA puede ayudar con explicación, scaffolding, refactoring, pruebas, datos sintéticos, documentación y debugging cuando la política lo permita.

No cargue código propietario, datos de clientes, secretos, credenciales o información no publicada en herramientas no aprobadas. Revise y pruebe el código generado. Verifique APIs/paquetes, seguridad y licencias. No

permita deploy autónomo fuera de gobernanza. El output de IA no es evidencia de ejecución.

Documentación y mantenibilidad

Documento propósito, decisiones, setup, configuración, APIs, modelos de datos, deployment, runbooks, límites, troubleshooting y ownership cuando sea útil. Prefiera cambios pequeños y revisables frente a complejidad innecesaria.

Límites éticos y profesionales

No se debe:

- fabricar resultados de pruebas o rendimiento;
- ocultar defectos de alto impacto;
- desplegar sin autoridad;
- evitar reviews o controles de seguridad;
- confirmar secretos;
- usar datos de producción en demos personales;
- debilitar autorización sin requisitos aprobados;
- explotar sistemas fuera de permiso;
- afirmar que un sistema es seguro/sin bugs solo porque las pruebas pasaron;
- publicar código o arquitectura privada del empleador.

Educación y entrada — Estados Unidos

O*NET ubica la ocupación en **Job Zone Four — Considerable Preparation Needed**. Las respuestas actuales de nuevas contrataciones son aproximadamente **85% bachelor's degree**, **5% associate degree** y **5% master's degree**. No son reglas universales para cada vacante.

CareerOneStop/American Job Centers ayudan a localizar WIOA y otras rutas. Elegibilidad y financiamiento varían.

O*NET lista títulos de Registered Apprenticeship como **Application Developer**, **Commercial Drone Software Developer**, **Devops Engineer (Nof)** y **Software Developer (Nof)**. Verifique aperturas reales en Apprenticeship.gov.

Canadá

Job Bank usa **NOC 21232 — Software developers and programmers**. Suele requerirse bachelor's degree en informática/software engineering u otra disciplina con programación significativa o college program relacionado.

Job Bank identifica actualmente la ocupación como **no regulada en Canadá**.

Salarios Canadá

- **C\$30.00/hora** bajo;
- **C\$48.08/hora** mediano;
- **C\$76.92/hora** alto.

Perspectiva Canadá

La demanda y oferta nacionales 2024-2033 se esperan ampliamente en equilibrio. Las perspectivas provinciales a tres años varían; verifique la ubicación.

Colombia

CUOC 25120 — Desarrolladores de software es correspondencia directa, nivel de competencia 4, e incluye análisis, diseño, desarrollo, pruebas, mantenimiento e implementación de soluciones.

No se fabrica un salario nacional representativo a partir de indicadores históricos/no representativos.

SENA

Análisis y desarrollo de software

- Tecnólogo;
- **3.984 horas**;
- formación titulada;
- requisitos, análisis, diseño, desarrollo, implementación y calidad;
- verifique cohorte, modalidad, cupos y admisión.

América Latina y Caribe

OIT/Cinterfor permite localizar instituciones nacionales de formación profesional. No garantiza cursos, becas, cupos o financiamiento.

Salarios y perspectiva

Estados Unidos oficial

Percentil	Anual	Por hora
10	\$82,460	\$39.64
25	\$105,210	\$50.58
Mediana	\$135,980	\$65.38
75	\$171,980	\$82.68
90	\$214,670	\$103.21

2024-2034:

- empleo 2024: **1,693,800**;
- proyectado 2034: **1,961,400**;
- crecimiento: **16%**;
- vacantes anuales: **115,200**.

Contexto no gubernamental adyacente

La URL de Indeed para Software Developer redirige actualmente a **Software Engineer**. La página, actualizada **10 de agosto de 2026**, muestra aproximadamente:

- promedio **\$135,356/año**;
- bajo **\$80,008/año**;
- alto **\$228,992/año**;
- **39.3 mil** observaciones;
- **36 meses** de ofertas;
- bonus en efectivo **\$5,000/año**.

Por la redirección, estos valores son solo contexto de mercado adyacente, no estadística exacta del título Software Developer ni fuente oficial.

Secuencia práctica

1. Fundamentos: un lenguaje, Git, debugging, pruebas y estructuras básicas.
2. Aplicaciones/datos: API o interfaz, persistencia, validación y errores.
3. Ingeniería: review, CI, logging, configuración/secrets y dependencias.
4. Producción: deployment, rollback, observabilidad, cloud, seguridad y rendimiento.
5. Especialización: aplicaciones, back end, móvil, plataforma, cloud, embebido, datos u otro dominio.

Portafolio seguro

Use software propio, open source, con licencia o demo y datos sintéticos/públicos. Muestre requisitos, decisiones de diseño, historial Git, APIs/datos, pruebas, configuración segura, CI, documentación y un deployment demo controlado.

No publique código del empleador, datos de clientes, credenciales, infraestructura privada, endpoints internos o vulnerabilidades sin autorización.

Plan de cuatro semanas

Semana 1

Elija un lenguaje, cree programas pequeños y use Git y unit tests.

Semana 2

Construya una aplicación/servicio con API/interfaz, persistencia, validación y manejo de errores.

Semana 3

Agregue integration tests, logging estructurado, configuración de ambientes y un pipeline CI simple.

Semana 4

Documente arquitectura, supuestos de seguridad y límites; cree una release demo controlada y bullets de currículum precisos.

Títulos de búsqueda

- Software Developer;
- Junior Software Developer;
- Software Engineer;
- Application Developer;
- Back-End Developer;
- Platform Developer;
- Cloud Developer;
- Systems Developer;
- Integration Developer;
- Java Developer;
- Python Developer;
- .NET Developer;
- Mobile Developer;
- DevOps Engineer cuando el desarrollo sea central.

Preguntas antes de aceptar un puesto

- ¿Qué software posee el equipo?
- ¿Qué lenguajes/frameworks se usan realmente?
- ¿Cómo se toman decisiones de requisitos y arquitectura?
- ¿Cómo funcionan code review y pruebas?
- ¿Cómo se aprueban y revierten releases?
- ¿Quién atiende incidentes/on-call?
- ¿Cómo gestionan secretos y dependencias?
- ¿Qué responsabilidades de seguridad/privacidad tiene desarrollo?

- ¿Cómo priorizan deuda técnica?
- ¿Qué distingue junior de senior?

Fuentes y enlaces de verificación

Estados Unidos

- O*NET details: [Source link](#)
- O*NET summary: [Source link](#)
- O*NET Job Zone: [Source link](#)
- O*NET wages: [Source link](#)
- O*NET outlook: [Source link](#)
- O*NET technologies: [Source link](#)
- CareerOneStop WIOA: [Source link](#)
- Apprenticeship.gov: <https://www.apprenticeship.gov/>
- Indeed contexto adyacente: [Source link](#)

Canadá

- Job Bank summary: [Source link](#)
- Job Bank requirements: [Source link](#)
- Job Bank wages: [Source link](#)
- Job Bank outlook: [Source link](#)
- Canada training: [Source link](#)

Colombia y América Latina

- CUOC 25120: [Source link](#)
- SENA Análisis y desarrollo de software: [Source link](#)
- OIT/Cinterfor: <https://www.oitcinterfor.org/statsfp/paises>

Seguridad, IA y accesibilidad

- NIST SSDF: <https://csrc.nist.gov/pubs/sp/800/218/final>
- OWASP WSTG: [Source link](#)
- NIST AI RMF: [Source link](#)
- Section 508: <https://www.section508.gov/create/>
- WCAG 2.2: <https://www.w3.org/TR/WCAG22/>

Aviso importante

Esta guía ofrece información general educativa y profesional. No garantiza empleo, ingresos, admisión, financiamiento, apprenticeship, certificación, ascenso, seguridad, accesibilidad ni otro resultado.

No se afirma certificación humana independiente, acreditación profesional, revisión legal, evaluación de seguridad, certificación de accesibilidad, certificación cloud/vendor ni traducción certificada salvo documentación separada.

Autor y asistencia de IA

Creada y dirigida por **Alberto “Al” Leiva**. ChatGPT apoyó investigación, organización, edición, traducción y preparación documental bajo la dirección del autor. El autor conserva la responsabilidad editorial y de publicación.

Licencia

Salvo indicación contraria, este material está licenciado bajo **CC BY-NC-SA 4.0**.